

1.1 Die Warteschlange (Queue)

In der Arztpraxis wird in der Reihenfolge der Ankunft aufgerufen. Das **Projekt Praxis** Website: *Station 1.1* verwaltet das Wartezimmer zunächst mit einem **Array**. Jedes Wartezimmer kann `add(Patient p)` (hinten anstellen), `poll()` (vorne aufrufen) und `ausgeben()`.

Aufgabe 1) Projekt Praxis

Starte das Hauptprogramm nacheinander mit Variante 1, 2 und 3. Notiere Idee und Problem.

Variante	Idee	Was geht schief?
1 Suchen		
2 Nummer		
3 Aufrücken		

Was passiert bei **allen drei** Varianten mit Mia? _____

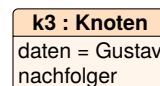
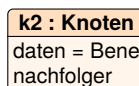
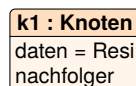
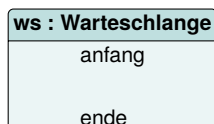
Probleme mit Arrays

Ein Array hat eine _____ und belegt schon beim Erzeugen _____ für alle Plätze, auch für leere (`null`). Zum Vergrößern müsste man _____ erzeugen und alles _____. Damit die Reihenfolge stimmt, müssen beim Herausnehmen alle _____.

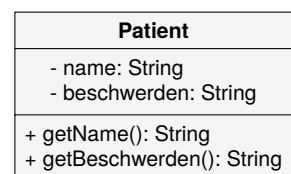
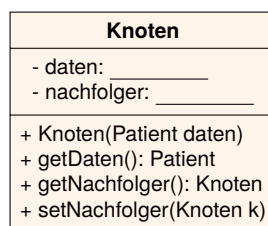
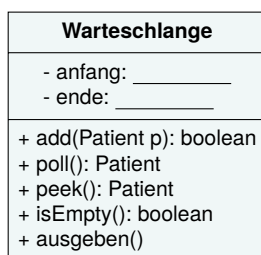
Idee: Jedes Element kennt zusätzlich _____. So entsteht eine **Kette** beliebiger Länge.

Aufgabe 2)

a) Resi, Bene und Gustav warten. Zeichne alle Verweise als Pfeile ein (oder `null`). Markiere, wo `poll()` herausnimmt und wo `add()` anhängt.



b) Ergänze die Datentypen und zeichne die Beziehungen mit Namen und Multiplizitäten ein.

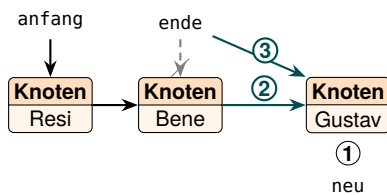


Die Warteschlange (Queue)

- Eine Warteschlange speichert _____ Objekte in einer festen _____.
- Die Warteschlange kennt nur den ersten (anfang) und letzten Knoten (ende), jeder Knoten seine daten und seinen _____. Beim letzten Knoten ist er _____.
- Eine Beziehung zwischen Objekten **derselben** Klasse heißt _____. Im Code wird sie zu einem Referenzattribut, dessen Typ die _____ ist: `private Knoten nachfolger;`
- `add` hängt _____ an, `poll` nimmt _____ weg: _____, First In – First Out.

Aufgabe 3) Projekt Praxis, TODO 3

add(Patient p) braucht drei Schritte. Übersetze jeden Schritt im Bild in eine Zeile Code.



- ① Neuen Knoten für p erzeugen.
- ② Bisher letzter Knoten bekommt neu als Nachfolger.
- ③ ende verweist auf den neuen Knoten.

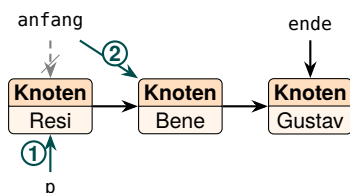
```
public boolean add(Patient p) {
    Knoten neu = _____; // ①
    if (isEmpty()) { // Sonderfall
        anfang = _____;
        ende = _____;
    } else {
        _____; // ②
        _____; // ③
    }
    return true;
}
```

b) Warum braucht man den Sonderfall für die leere Schlange? Was passiert ohne ihn?

c) Übertrage die Methode ins Projekt und teste sie mit new Warteschlange().

Aufgabe 4) Projekt Praxis, TODO 4

poll() holt den vordersten Patienten heraus und gibt ihn zurück.



- ① Die Daten des ersten Knotens in p merken.
- ② anfang rückt auf den Nachfolger weiter.
- ③ War das der letzte Knoten, wird auch ende leer.
- ④ p zurückgeben.

```
public Patient poll() {
    if (isEmpty()) {
        return _____; // niemand wartet
    } else {
        Patient p = _____; // ①
        anfang = _____; // ②
        if (_____ ) { // ③
            ende = null;
        }
        return _____; // ④
    }
}
```

b) Warum muss Schritt 1 vor Schritt 2 stehen?

c) Was geschieht danach mit dem Knoten von Resi?

Aufgabe 5) Projekt Praxis

Stelle das Hauptprogramm auf new Warteschlange() um und vergleiche mit Variante 3.

	Variante 3 (Aufrücken)	Warteschlange
Verschiebungen bei poll(), wenn n warten		
Wie viele können warten?		

Es geht auch anders – besprecht mündlich, warum man es so machen kann:

- Die Warteschlange merkt sich **nur anfang**, kein ende. Wie findet add dann den letzten Knoten? Was wird einfacher, was langsamer?
- poll() gibt den **Knoten** statt des Patienten zurück. Warum ist es besser, wenn das Hauptprogramm die Klasse Knoten gar nicht kennen muss?
- Patient bekommt **selbst** ein Attribut nachfolger, die Klasse Knoten entfällt. Was spart man, was verliert man?
- add liefert boolean, obwohl hier immer true herauskommt. Warum ist das trotzdem sinnvoll? (Tipp: LinkedList in der Breitensuche, Klasse 11)